

### Example 1: Redundant Normalisation

**Paper Claim:** "Input images are normalised to using max scaling."

**Code Section:**

*# Scales to [0, 1] but adds redundant operation*

```
images = images / 255.0  
images = images * 1.0 # No effect
```

**Affects Results?** No

**Explanation:** The second operation is mathematically neutral ( $\times 1.0$ ), so normalisation remains valid.

### Example 2: Undocumented Layer Name

**Paper Claim:** "A ResNet-18 architecture is used for feature extraction."

**Code Section:**

```
model = ResNet(block=BasicBlock, layers=[2, 2, 2, 2], name="backbone")
```

**Affects Results?** No

**Explanation:** Layer naming conventions don't alter model functionality or performance.

### Example 3: Silent Batch Size Mismatch

**Paper Claim:** "Models are trained with a batch size of 64."

**Code Section:**

```
BATCH_SIZE = 64  
train_loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
```

```
# Later in training loop:  
for X, y in train_loader:  
    X, y = X.to(device), y.to(device)  
    # Paper doesn't mention gradient accumulation  
    outputs = model(X)  
    loss = criterion(outputs, y)  
    loss.backward()  
    if (i+1) % 2 == 0: # Effective batch size = 128  
        optimiser.step()  
        optimiser.zero_grad()
```

**Affects Results?** Yes

**Explanation:** Gradient accumulation alters effective batch size ( $64 \rightarrow 128$ ), affecting optimisation dynamics.

### Example 4: Redundant Identity Layer

**Paper Claim:** "The transformer uses 8 attention heads."

**Code Section:**

```
# Extra identity layer that does nothing
class TransformerBlock(nn.Module):
    def __init__(self):
        super().__init__()
        self.attn = MultiheadAttention(embed_dim=512, num_heads=8)
        self.norm = nn.Identity() # Paper doesn't mention this
```

**Affects Results? No**

**Explanation:** Identity layers don't alter model behavior despite being undocumented.

**Example 5: Wrong Value**

**Paper Claim:** "Gradients are clipped at 1.0 during training."

**Code Section:**

```
torch.nn.utils.clip_grad_value_(model.parameters(), 0.1) # Paper claims 1.0
```

**Affects Results? Yes**

**Explanation:** Aggressive clipping (0.1 vs. 1.0) stifles gradient updates, altering training dynamics.

**Example 6: System Architecture**

**Paper claim:** "We used a convolutional neural network with 3 convolutional layers and 2 fully connected layers."

**Code section:**

```
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation='relu'),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
```

**Affects results?: No**

**Explanation:** The overall architecture matches the paper's description. These minor deviations are unlikely to significantly affect the results, as the fundamental methodology remains the same, unless the paper mentions specific values for each layer.

**Example 7: Batch Size**

**Paper claim:** "We used a batch size of 64 for training."

**Code section:** batch\_size = 64

**Affects results: No**

**Explanation:** The code uses the same batch size as mentioned in the paper. There is no discrepancy here, and the results should be reproducible.

**Example 8:**

**Paper claim:** "The input images were normalised to have zero mean and unit variance."

**Code section:** # No normalisation code found

**Affects results:** Yes

**Explanation:** The paper claims that the input images were normalised, but there is no corresponding code for normalisation. This discrepancy could lead to different input data distributions and affect the model's performance, making the results difficult to reproduce.